```
import pandas as pd
df = pd.read_csv('/content/car_evaluation.csv')
df.head()
```

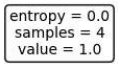
	vhigh	vhigh.1	2	2.1	small	low	unacc
0	vhigh	vhigh	2	2	small	med	unacc
1	vhigh	vhigh	2	2	small	high	unacc
2	vhigh	vhigh	2	2	med	low	unacc
3	vhigh	vhigh	2	2	med	med	unacc
4	vhigh	vhigh	2	2	med	high	unacc

Next steps: Generate code with df | New interactive sheet

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
import matplotlib.pyplot as plt

# ------------------------------------------------
# 1. Create CLEAN dataset
# ------------------------------------------------

data = [
    ["vhigh", "vhigh.1", "2", "2.1", "small", "low",  "unacc"],
    ["vhigh", "vhigh",   "2", "2",   "small", "med",  "unacc"],
    ["vhigh", "vhigh",   "2", "2",   "small", "high", "unacc"],
    ["vhigh", "vhigh",   "2", "2",   "med",   "low",  "unacc"],
    ["vhigh", "vhigh",   "2", "2",   "med",   "med",  "unacc"],
    ["vhigh", "vhigh",   "2", "2",   "med",   "high", "unacc"]
]

columns = ["buying", "maint", "doors", "persons", "lug_boot", "safety", "class"]

df = pd.DataFrame(data, columns=columns)

# ------------------------------------------------
# 2. Encode labels
# ------------------------------------------------
le = LabelEncoder()
for col in df.columns:
    df[col] = le.fit_transform(df[col])

# ------------------------------------------------
# 3. Split dataset
# ------------------------------------------------
X = df.drop("class", axis=1)
y = df["class"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# ------------------------------------------------
# 4. Train Decision Tree
# ------------------------------------------------
model = DecisionTreeClassifier(criterion="entropy", random_state=42)
model.fit(X_train, y_train)

# ------------------------------------------------
# 5. Accuracy
# ------------------------------------------------
accuracy = model.score(X_test, y_test)
print("Model Accuracy:", accuracy)

# ------------------------------------------------
# 6. Show Decision Rules (Text Form)
# ------------------------------------------------
```

```
rules = tree.export_text(model, feature_names=list(X.columns))
print("\nDecision Rules:\n")
print(rules)

# ------------------------------------------------
# 7. Plot the Tree
# ------------------------------------------------
plt.figure(figsize=(16, 10))
tree.plot_tree(
    model,
    feature_names=X.columns,
    class_names=["unacc"],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.show()
```

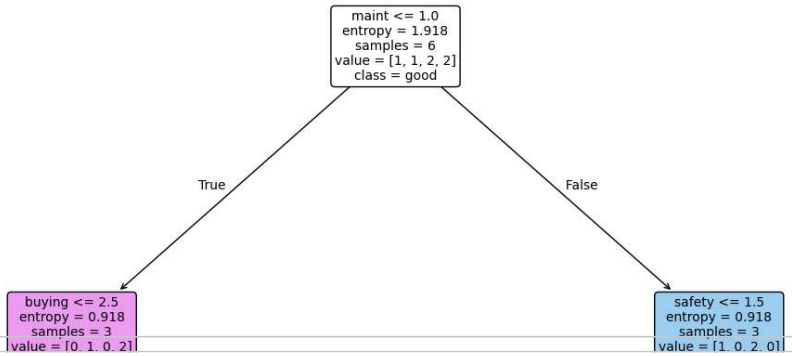
```
Model Accuracy: 1.0

Decision Rules:

|--- class: 0
```




```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
import matplotlib.pyplot as plt

# ------------------------------------------------------
# 1. Dataset (balanced so the tree actually splits)
# ------------------------------------------------------
data = [
    ["vhigh","vhigh","2","2","small","low","unacc"],
    ["vhigh","vhigh","2","4","small","med","acc"],
    ["vhigh","high","4","4","med","high","good"],
    ["high","high","4","4","big","high","vgood"],
    ["med","med","4","4","med","med","acc"],
    ["low","low","2","2","small","low","unacc"],
    ["med","high","4","more","big","high","vgood"],
```

```
    ["low","med","3","4","med","med","good"],
    ["high","med","4","more","big","low","unacc"]
]

columns = ["buying","maint","doors","persons","lug_boot","safety","class"]
df = pd.DataFrame(data, columns=columns)

# ----------------------------------------------------
# 2. Encode categorical data
# ----------------------------------------------------
le = LabelEncoder()
for col in df.columns:
    df[col] = le.fit_transform(df[col])

# ----------------------------------------------------
# 3. Split dataset
# ----------------------------------------------------
X = df.drop("class", axis=1)
y = df["class"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# ----------------------------------------------------
# 4. Build Decision Tree Model
# ----------------------------------------------------
model = DecisionTreeClassifier(criterion="entropy", max_depth=5, random_state=42)
model.fit(X_train, y_train)

# ----------------------------------------------------
# 5. Accuracy
# ----------------------------------------------------
accuracy = model.score(X_test, y_test)
print("Model Accuracy:", accuracy)

# ----------------------------------------------------
# 6. Decision Rules
# ----------------------------------------------------
rules = export_text(model, feature_names=list(X.columns))
print("\nDecision Tree Rules:\n")
print(rules)

# ----------------------------------------------------
# 7. Plot the Tree
# ----------------------------------------------------
plt.figure(figsize=(18, 12))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=["unacc", "acc", "good", "vgood"],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.show()
```

```
Model Accuracy: 0.3333333333333333

Decision Tree Rules:

|--- maint <= 1.00
|   |--- buying <= 2.50
|   |   |--- class: 3
|   |--- buying >  2.50
|   |   |--- class: 1
|--- maint >  1.00
|   |--- safety <= 1.50
|   |   |--- class: 2
|   |--- safety >  1.50
|   |   |--- class: 0
```




```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

df = pd.read_csv('/content/car_evaluation.csv')

# Rename last column
df = df.rename(columns={df.columns[-1]: "class"})

# Encode
for col in df.columns:
    df[col] = LabelEncoder().fit_transform(df[col])

X = df.drop("class", axis=1)
y = df["class"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

rf = RandomForestClassifier(n_estimators=200, criterion="entropy", random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

accuracy_score(y_test, y_pred), classification_report(y_test, y_pred)
```

```
(0.9672447013487476,
 '              precision    recall  f1-score   support\n\n           0       0.96      0.92      0.94       118\n           1
0.92      0.71      0.80        17\n           2       0.98      1.00      0.99       361\n           3       0.81      0.91
0.86        23\n\n    accuracy                           0.97       519\n   macro avg       0.92      0.88      0.90
519\nweighted avg       0.97      0.97      0.97       519\n')
```

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

# ------------------------------------------------------
# 1. Dataset (same balanced dataset from decision tree)
# ------------------------------------------------------
data = [
    ["vhigh","vhigh","2","2","small","low","unacc"],
```

```
    ["vhigh","vhigh","2","4","small","med","acc"],
    ["vhigh","high","4","4","med","high","good"],
    ["high","high","4","4","big","high","vgood"],
    ["med","med","4","4","med","med","acc"],
    ["low","low","2","2","small","low","unacc"],
    ["med","high","4","more","big","high","vgood"],
    ["low","med","3","4","med","med","good"],
    ["high","med","4","more","big","low","unacc"]
]

columns = ["buying","maint","doors","persons","lug_boot","safety","class"]

df = pd.DataFrame(data, columns=columns)

# ----------------------------------------------------
# 2. Label Encoding
# ----------------------------------------------------
le = LabelEncoder()
for col in df.columns:
    df[col] = le.fit_transform(df[col])

# ----------------------------------------------------
# 3. Split dataset
# ----------------------------------------------------
X = df.drop("class", axis=1)
y = df["class"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# ----------------------------------------------------
# 4. Random Forest Model
# ----------------------------------------------------
rf = RandomForestClassifier(
    n_estimators=200,
    criterion="entropy",
    max_depth=6,
    random_state=42
)

rf.fit(X_train, y_train)

# ----------------------------------------------------
# 5. Predictions + Accuracy
# ----------------------------------------------------
y_pred = rf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# ----------------------------------------------------
# 6. Feature Importance
# ----------------------------------------------------
importances = pd.Series(rf.feature_importances_, index=X.columns)
print("\nFeature Importances:\n")
print(importances.sort_values(ascending=False))
```

```
Accuracy: 0.3333333333333333

Classification Report:

              precision    recall  f1-score   support

           0       0.00      0.00      0.00         1
           1       0.00      0.00      0.00         1
           2       0.50      1.00      0.67         1

    accuracy                           0.33         3
   macro avg       0.17      0.33      0.22         3
weighted avg       0.17      0.33      0.22         3


Feature Importances:

safety      0.282215
```

```
maint       0.232528
lug_boot    0.187282
buying      0.165277
persons     0.106987
doors       0.025711
dtype: float64
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
Start coding or generate with AI.
```